

Explications Théoriques - Administration Unix/Linux

Introduction

Ce document présente les explications théoriques et conceptuelles de l'administration Unix/Linux. Contrairement au résumé pratique, ce document se concentre sur la compréhension des mécanismes internes, des principes architecturaux et des concepts fondamentaux qui sous-tendent le fonctionnement des systèmes Unix/Linux.

Table des matières

1	Architecture et Principes Fondamentaux	3
1.1	Philosophie Unix	3
1.2	Architecture en Couches	3
1.2.1	Couche Matérielle	3
1.2.2	Couche Noyau (Kernel)	3
1.2.3	Couche Système	4
1.2.4	Couche Utilisateur	4
1.3	Mode Noyau vs Mode Utilisateur	4
2	Gestion de la Mémoire	4
2.1	Architecture de la Mémoire Virtuelle	4
2.1.1	Pages et Pagination	4
2.1.2	Translation d'Adresses	5
2.2	Swap et Mémoire Virtuelle	5
2.3	Gestion de la Mémoire par Processus	5
3	Gestion des Processus	6
3.1	Concept de Processus	6
3.2	Cycle de Vie d'un Processus	6
3.2.1	Création (Fork)	6
3.2.2	Exécution (Exec)	6
3.2.3	États d'un Processus	6
3.3	Planification des Processus	7
3.3.1	Priorités et Nice	7
3.3.2	Algorithmes de Planification	7
3.4	Communication Inter-Processus	7
3.4.1	Signaux	7
3.4.2	Pipes	8
3.4.3	Shared Memory	8
4	Systèmes de Fichiers	8
4.1	Concept de Système de Fichiers	8
4.2	Structure des Systèmes de Fichiers	8
4.2.1	Inodes	8
4.2.2	Blocs	9
4.2.3	Répertoires	9
4.3	Liens	9
4.3.1	Liens Physiques (Hard Links)	9
4.3.2	Liens Symboliques (Soft Links)	9
4.4	Types de Systèmes de Fichiers	10
4.4.1	ext2/ext3/ext4	10
4.4.2	XFS	10
4.5	Montage et Points de Montage	10

5	Sécurité et Permissions	10
5.1	Modèle de Permissions Unix	10
5.1.1	Utilisateurs et Groupes	10
5.1.2	Permissions Discretionnaires (DAC)	11
5.1.3	Droits Spéciaux	11
5.2	ACLs (Access Control Lists)	11
5.3	SELinux : Contrôle d'Accès Obligatoire	11
5.3.1	Principe MAC (Mandatory Access Control)	11
5.3.2	Contexte SELinux	12
5.3.3	Modes SELinux	12
6	Démarrage du Système	12
6.1	Séquence de Démarrage	12
6.1.1	Phase 1 : Matériel et Firmware	12
6.1.2	Phase 2 : Bootloader	12
6.1.3	Phase 3 : Noyau	13
6.1.4	Phase 4 : Espace Utilisateur	13
6.2	Initramfs (Initial RAM Filesystem)	13
6.3	Systemd : Gestionnaire de Services Moderne	13
6.3.1	Concepts Fondamentaux	13
6.3.2	Targets vs Runlevels	14
7	Gestion des Paquets	14
7.1	Concept de Paquetage	14
7.2	Gestion des Dépendances	14
7.3	Formats de Paquets	14
7.3.1	RPM (RedHat Package Manager)	14
7.3.2	DEB (Debian Package)	15
8	Réseaux et Communication	15
8.1	Modèle TCP/IP	15
8.2	Sockets	15
8.3	Interfaces Réseau	15

1 Architecture et Principes Fondamentaux

1.1 Philosophie Unix

La philosophie Unix repose sur plusieurs principes fondamentaux qui ont façonné le design des systèmes Unix et Linux :

- **Simplicité** : Chaque programme doit faire une seule chose et la faire bien. Cette approche modulaire facilite la maintenance et la compréhension du système.
- **Composition** : Les programmes simples peuvent être combinés via des pipes et des redirections pour créer des solutions complexes. Cette approche favorise la réutilisabilité.
- **Textualité** : Les fichiers de configuration et les interfaces sont basés sur du texte, ce qui permet une manipulation facile avec des outils standards et une traçabilité claire.
- **Transparence** : Le système expose ses mécanismes internes de manière accessible, permettant aux administrateurs de comprendre et de diagnostiquer les problèmes.
- **Portabilité** : L'écriture en C et l'abstraction des détails matériels permettent une portabilité entre différentes architectures.

1.2 Architecture en Couches

Le système Unix/Linux est organisé en couches distinctes, chacune ayant des responsabilités spécifiques :

1.2.1 Couche Matérielle

La couche matérielle comprend tous les composants physiques : processeurs, mémoire (RAM), disques de stockage, cartes réseau, et autres périphériques. Cette couche est gérée directement par le noyau via des pilotes de périphériques (drivers).

1.2.2 Couche Noyau (Kernel)

Le noyau est le cœur du système d'exploitation. Il agit comme un intermédiaire entre le matériel et les logiciels applicatifs. Ses responsabilités principales incluent :

- **Gestion de la mémoire** : Allocation et libération de la mémoire pour les processus, gestion de la mémoire virtuelle, pagination et swapping.
- **Gestion des processus** : Création, planification, synchronisation et terminaison des processus. Le noyau maintient une table des processus et gère le partage du temps CPU.
- **Gestion des fichiers** : Abstraction des systèmes de fichiers, gestion des inodes, cache des fichiers, et interface avec les périphériques de stockage.
- **Gestion des périphériques** : Communication avec le matériel via des pilotes, gestion des interruptions matérielles, et abstraction des périphériques en fichiers spéciaux.
- **Sécurité** : Contrôle d'accès basé sur les permissions, gestion des utilisateurs et groupes, et implémentation de mécanismes de sécurité avancés comme SELinux.

1.2.3 Couche Système

Cette couche comprend les bibliothèques système (comme la bibliothèque C standard, libc) et les services système qui fournissent des fonctionnalités communes aux applications. Les appels système (system calls) permettent aux applications d'interagir avec le noyau de manière contrôlée.

1.2.4 Couche Utilisateur

La couche utilisateur contient les applications, les shells, et les outils utilisateur. Cette couche s'exécute en mode utilisateur, avec des privilèges limités, et doit passer par le noyau pour accéder aux ressources système.

1.3 Mode Noyau vs Mode Utilisateur

Une distinction fondamentale dans l'architecture Unix/Linux est la séparation entre le mode noyau (kernel mode) et le mode utilisateur (user mode) :

- **Mode Noyau** : Le noyau s'exécute avec des privilèges complets, peut accéder directement au matériel, et peut modifier les structures de données critiques du système. Toute erreur dans le noyau peut provoquer un crash système.
- **Mode Utilisateur** : Les applications s'exécutent avec des privilèges limités. Elles ne peuvent pas accéder directement au matériel et doivent utiliser les appels système pour demander des services au noyau. Cette séparation protège le système contre les erreurs des applications.
- **Transition** : Le passage du mode utilisateur au mode noyau se fait via des interruptions matérielles, des exceptions, ou des appels système explicites. Cette transition est coûteuse en performance, ce qui explique pourquoi le noyau essaie de minimiser ces transitions.

2 Gestion de la Mémoire

2.1 Architecture de la Mémoire Virtuelle

La mémoire virtuelle est un mécanisme fondamental qui permet au système de donner à chaque processus l'illusion d'avoir accès à un espace d'adressage linéaire complet, indépendamment de la quantité de mémoire physique disponible.

2.1.1 Pages et Pagination

La mémoire est divisée en pages de taille fixe (généralement 4 Ko sur les systèmes x86-64). Le système de pagination permet de :

- **Virtualiser la mémoire** : Chaque processus voit un espace d'adressage virtuel continu, même si la mémoire physique est fragmentée.
- **Protéger les processus** : Les pages peuvent avoir des permissions (lecture, écriture, exécution) qui empêchent un processus d'accéder à la mémoire d'un autre processus.
- **Optimiser l'utilisation** : Les pages non utilisées peuvent être stockées sur disque (swap), libérant la RAM pour d'autres processus.

2.1.2 Translation d'Adresses

L'Unité de Gestion Mémoire (MMU) du processeur traduit les adresses virtuelles en adresses physiques. Cette traduction utilise des tables de pages maintenues par le noyau. Le processus de traduction comprend :

1. Le processeur génère une adresse virtuelle lors d'un accès mémoire.
2. La MMU consulte la table des pages pour trouver la page physique correspondante.
3. Si la page est en RAM, l'accès se fait directement.
4. Si la page est sur disque (page fault), le noyau la charge en RAM et met à jour la table des pages.

2.2 Swap et Mémoire Virtuelle

Le swap est un mécanisme qui étend la mémoire RAM disponible en utilisant l'espace disque. Voici comment cela fonctionne :

- **Pages inactives** : Le noyau identifie les pages de mémoire qui n'ont pas été utilisées récemment.
- **Écriture sur disque** : Ces pages sont écrites dans l'espace swap sur le disque.
- **Libération de RAM** : Les pages physiques sont libérées pour être utilisées par d'autres processus.
- **Page fault** : Lorsqu'un processus tente d'accéder à une page qui a été swapée, une interruption (page fault) se produit.
- **Chargement** : Le noyau charge la page depuis le swap vers la RAM, éventuellement en swapant une autre page si la RAM est pleine.

Cette mécanique permet au système d'exécuter plus de processus que la RAM physique ne le permettrait, au prix d'une performance réduite lorsque le swapping est fréquent (thrashing).

2.3 Gestion de la Mémoire par Processus

Chaque processus a son propre espace d'adressage virtuel, organisé en plusieurs segments :

- **Segment texte** : Contient le code exécutable du programme. Il est en lecture seule et peut être partagé entre plusieurs instances du même programme.
- **Segment données** : Contient les variables globales et statiques initialisées.
- **Segment BSS** : Contient les variables globales non initialisées (Block Started by Symbol).
- **Heap (tas)** : Zone de mémoire dynamique allouée par `malloc()` et libérée par `free()`. Le heap grandit vers les adresses hautes.
- **Stack (pile)** : Zone de mémoire pour les variables locales et les appels de fonction. Le stack grandit vers les adresses basses. Chaque thread a son propre stack.

3 Gestion des Processus

3.1 Concept de Processus

Un processus est une instance d'un programme en cours d'exécution. Il représente l'unité fondamentale d'exécution dans un système Unix/Linux. Chaque processus possède :

- **Un espace d'adressage mémoire** : Isolé des autres processus pour la sécurité.
- **Des ressources système** : Descripteurs de fichiers ouverts, variables d'environnement, signaux en attente.
- **Un contexte d'exécution** : Registres CPU, compteur de programme, état de la pile.
- **Des métadonnées** : PID (Process ID), PPID (Parent Process ID), UID, GID, priorité.

3.2 Cycle de Vie d'un Processus

Le cycle de vie d'un processus suit plusieurs étapes :

3.2.1 Création (Fork)

La création d'un processus se fait via l'appel système `fork()`. Ce mécanisme crée une copie exacte du processus appelant :

- Le processus parent et l'enfant partagent initialement le même espace mémoire (copy-on-write).
- L'enfant hérite des descripteurs de fichiers ouverts du parent.
- L'enfant reçoit un nouveau PID unique.
- Le retour de `fork()` est différent dans le parent (PID de l'enfant) et l'enfant (0).

3.2.2 Exécution (Exec)

L'appel système `exec()` remplace l'image mémoire du processus par un nouveau programme :

- Le code et les données du processus sont remplacés.
- Les descripteurs de fichiers ouverts sont préservés (sauf ceux marqués `close-on-exec`).
- Le PID et les relations parent-enfant restent inchangés.
- Les variables d'environnement peuvent être modifiées selon la variante d'`exec` utilisée.

3.2.3 États d'un Processus

Un processus peut être dans plusieurs états :

- **Running (R)** : Le processus est en cours d'exécution ou prêt à s'exécuter. Il attend son tour sur le CPU.
- **Sleeping (S)** : Le processus attend un événement (entrée/sortie, signal, verrou). Il est interruptible et peut être réveillé.

- **Disk Sleep (D)** : Le processus attend une opération disque critique. Il est non-interruptible pour éviter la corruption des données.
- **Stopped (T)** : Le processus a été suspendu par un signal (SIGSTOP, SIGTSTP). Il peut être repris avec SIGCONT.
- **Zombie (Z)** : Le processus est terminé mais son parent n'a pas encore récupéré son statut de sortie. Les ressources sont libérées sauf l'entrée dans la table des processus.

3.3 Planification des Processus

Le planificateur (scheduler) du noyau décide quel processus doit s'exécuter à un moment donné. Cette décision est basée sur plusieurs facteurs :

3.3.1 Priorités et Nice

Chaque processus a une priorité qui influence son temps CPU :

- La valeur **nice** va de -20 (priorité maximale) à +19 (priorité minimale).
- Les processus avec une nice négative ont plus de chances d'être exécutés.
- Seul root peut utiliser des valeurs nice négatives.
- Le planificateur combine la valeur nice avec d'autres facteurs (temps d'attente, type de processus) pour déterminer la priorité dynamique.

3.3.2 Algorithmes de Planification

Linux utilise le Completely Fair Scheduler (CFS) qui :

- Assure une distribution équitable du temps CPU entre tous les processus.
- Utilise une red-black tree pour maintenir les processus triés par temps CPU virtuel.
- Ajuste dynamiquement les quanta de temps selon la charge système.
- Donne la priorité aux processus interactifs pour améliorer la réactivité.

3.4 Communication Inter-Processus

Les processus peuvent communiquer de plusieurs manières :

3.4.1 Signaux

Les signaux sont des notifications asynchrones envoyées à un processus :

- **Asynchrone** : Le signal peut arriver à tout moment pendant l'exécution.
- **Pré-défini** : Chaque signal a un numéro et une action par défaut.
- **Handler** : Les processus peuvent définir des handlers personnalisés pour intercepter les signaux.
- **Blocage** : Certains signaux peuvent être bloqués temporairement.

3.4.2 Pipes

Les pipes permettent la communication unidirectionnelle entre processus :

- **Pipe anonyme** : Créé avec `pipe()`, permet la communication entre processus parent-enfant.
- **Pipe nommé (FIFO)** : Créé avec `mkfifo()`, permet la communication entre processus non liés.
- **Buffer limité** : Les pipes ont une taille de buffer limitée, ce qui peut bloquer l'écriture si le buffer est plein.

3.4.3 Shared Memory

La mémoire partagée permet à plusieurs processus d'accéder à la même région mémoire :

- **Performance** : Plus rapide que les pipes car pas de copie de données.
- **Synchronisation** : Nécessite des mécanismes de synchronisation (sémaphores, mutex) pour éviter les conditions de course.
- **IPC System V** : Utilise `shmget()`, `shmat()` pour créer et attacher la mémoire partagée.

4 Systèmes de Fichiers

4.1 Concept de Système de Fichiers

Un système de fichiers est une méthode d'organisation et de stockage des données sur un support de stockage. Il fournit une abstraction qui permet de manipuler les données comme une hiérarchie de fichiers et de répertoires, indépendamment des détails physiques du stockage.

4.2 Structure des Systèmes de Fichiers

4.2.1 Inodes

L'inode (index node) est une structure de données fondamentale qui contient toutes les métadonnées d'un fichier :

- **Type** : Fichier régulier, répertoire, lien symbolique, périphérique, etc.
- **Permissions** : Droits d'accès pour propriétaire, groupe, et autres.
- **Propriétaire** : UID et GID du propriétaire.
- **Taille** : Taille du fichier en octets.
- **Dates** : atime (dernier accès), mtime (dernière modification), ctime (dernier changement de métadonnées).
- **Pointeurs vers les blocs** : Adresses des blocs de données sur le disque.
- **Compteur de références** : Nombre de liens physiques pointant vers cet inode.

Important : Le nom du fichier n'est PAS stocké dans l'inode. Il est stocké dans le répertoire parent, qui associe un nom à un numéro d'inode.

4.2.2 Blocs

Les données des fichiers sont stockées en blocs de taille fixe (généralement 4 Ko). Le système de fichiers gère l'allocation des blocs :

- **Allocation** : Le système de fichiers maintient une bitmap ou une liste de blocs libres.
- **Fragmentation** : Les fichiers peuvent être fragmentés (blocs non contigus), ce qui peut affecter les performances.
- **Blocs indirects** : Pour les gros fichiers, les inodes utilisent des pointeurs indirects, doublement indirects, et triplement indirects pour référencer les blocs de données.

4.2.3 Répertoires

Un répertoire est un fichier spécial qui contient une liste d'associations nom-inode :

- Chaque entrée associe un nom de fichier à un numéro d'inode.
- Les répertoires "." et ".." sont des liens spéciaux vers le répertoire courant et le parent.
- La recherche dans un répertoire nécessite de parcourir les entrées jusqu'à trouver le nom correspondant.

4.3 Liens

4.3.1 Liens Physiques (Hard Links)

Un lien physique est une entrée de répertoire qui pointe vers le même inode qu'un autre nom :

- **Partage d'inode** : Plusieurs noms peuvent pointer vers le même inode.
- **Compteur de références** : L'inode maintient un compteur du nombre de liens.
- **Suppression** : Un fichier n'est réellement supprimé que lorsque le dernier lien est supprimé (compteur = 0).
- **Limitation** : Les liens physiques ne peuvent exister que dans le même système de fichiers.

4.3.2 Liens Symboliques (Soft Links)

Un lien symbolique est un fichier spécial qui contient le chemin vers un autre fichier :

- **Inode séparé** : Le lien symbolique a son propre inode.
- **Contenu** : Le contenu du lien est le chemin (absolu ou relatif) vers le fichier cible.
- **Transversal** : Peut pointer vers des fichiers dans d'autres systèmes de fichiers.
- **Orphelin** : Si le fichier cible est supprimé, le lien devient "cassé" (dangling link).

4.4 Types de Systèmes de Fichiers

4.4.1 ext2/ext3/ext4

La famille ext (Extended File System) est la plus commune sur Linux :

- **ext2** : Système de fichiers classique sans journalisation. Robuste mais nécessite fsck après un crash.
- **ext3** : Ajout de la journalisation à ext2. La journalisation enregistre les modifications avant de les appliquer, permettant une récupération rapide après un crash.
- **ext4** : Amélioration de ext3 avec support de volumes > 16 To, timestamps nanoseconde, allocation retardée (delayed allocation), et extents pour réduire la fragmentation.

4.4.2 XFS

XFS est un système de fichiers haute performance conçu pour les gros fichiers :

- **Journalisation** : Journalisation métadonnées pour récupération rapide.
- **Extents** : Allocation par extents (plages contiguës) plutôt que par blocs individuels.
- **Quotas** : Support natif des quotas utilisateur et groupe.
- **Snapshots** : Support des snapshots pour sauvegardes cohérentes.

4.5 Montage et Points de Montage

Le montage est le processus qui attache un système de fichiers à un point dans l'arborescence des répertoires :

- **Point de montage** : Répertoire où le système de fichiers est attaché.
- **Racine** : Le système de fichiers racine (/) est monté au démarrage.
- **Propagation** : Les systèmes de fichiers montés masquent le contenu du répertoire de montage.
- **Options** : Les options de montage (ro, rw, noexec, nosuid) contrôlent le comportement du système de fichiers monté.

5 Sécurité et Permissions

5.1 Modèle de Permissions Unix

Le modèle de sécurité Unix repose sur trois concepts fondamentaux :

5.1.1 Utilisateurs et Groupes

- **Utilisateur** : Chaque utilisateur a un identifiant unique (UID). L'utilisateur root (UID 0) a des privilèges complets.
- **Groupe** : Les utilisateurs appartiennent à un ou plusieurs groupes. Chaque groupe a un identifiant (GID).
- **Groupe primaire** : Chaque utilisateur a un groupe primaire, défini dans `/etc/passwd`.
- **Groupes secondaires** : Les utilisateurs peuvent appartenir à plusieurs groupes supplémentaires, définis dans `/etc/group`.

5.1.2 Permissions Discretionnaires (DAC)

Le Discretionary Access Control est le modèle de permissions classique :

- **Propriétaire** : Le créateur du fichier en est le propriétaire.
- **Trois catégories** : Permissions pour le propriétaire (u), le groupe (g), et les autres (o).
- **Trois droits** : Lecture (r), écriture (w), exécution (x).
- **Discretionnaire** : Le propriétaire peut modifier les permissions à sa discrétion.

5.1.3 Droits Spéciaux

Trois bits spéciaux étendent le modèle de base :

- **Setuid** : Permet à un processus d'hériter des privilèges du propriétaire du fichier exécutable, même s'il est lancé par un autre utilisateur. Risque de sécurité si mal utilisé.
- **Setgid** : Sur un fichier, permet l'exécution avec les privilèges du groupe. Sur un répertoire, force les nouveaux fichiers à appartenir au groupe du répertoire.
- **Sticky bit** : Sur un répertoire, empêche les utilisateurs non-propriétaires de supprimer ou renommer des fichiers dont ils ne sont pas propriétaires. Essentiel pour /tmp.

5.2 ACLs (Access Control Lists)

Les ACLs étendent le modèle de permissions classique :

- **Limitation du modèle classique** : Le modèle u/g/o ne permet qu'une seule entrée par catégorie.
- **ACLs** : Permettent de définir des permissions spécifiques pour plusieurs utilisateurs et groupes sur le même fichier.
- **Masque** : Le masque ACL limite les permissions effectives pour les groupes et les ACLs utilisateur, garantissant que les permissions ne dépassent pas le masque.
- **ACLs par défaut** : Sur les répertoires, les ACLs par défaut sont héritées par les nouveaux fichiers créés dans ce répertoire.

5.3 SELinux : Contrôle d'Accès Obligatoire

SELinux implémente un modèle de sécurité plus strict que le DAC :

5.3.1 Principe MAC (Mandatory Access Control)

- **Obligatoire** : Les politiques de sécurité sont définies par l'administrateur système et ne peuvent pas être contournées par les utilisateurs, même root.
- **Type Enforcement** : Chaque processus et ressource a un type (ou contexte). Les règles définissent quels types de processus peuvent accéder à quels types de ressources.
- **Isolation** : Même si un processus est compromis, SELinux limite les dommages en restreignant les ressources accessibles selon le type du processus.

5.3.2 Contexte SELinux

Chaque ressource (fichier, processus, port réseau) a un contexte SELinux de la forme :
`user:role:type:level`

- **User** : Identité SELinux (`system_u`, `user_u`, etc.)
- **Role** : Rôle dans la politique (`object_r` pour les objets, `system_r` pour les processus système)
- **Type** : Le composant le plus important. Détermine les permissions. Exemples :
 - `httpd_t` : Type pour les processus Apache
 - `httpd_sys_content_t` : Type pour les fichiers web accessibles par Apache
 - `shadow_t` : Type pour les fichiers de mots de passe
- **Level** : Niveau de sensibilité (pour Multi-Level Security)

5.3.3 Modes SELinux

- **Enforcing** : Mode strict où les violations sont bloquées et enregistrées. Utilisé en production.
- **Permissive** : Mode de débogage où les violations sont enregistrées mais pas bloquées. Permet d'identifier les problèmes sans interrompre le fonctionnement.
- **Disabled** : SELinux est complètement désactivé. Nécessite un redémarrage pour être réactivé.

6 Démarrage du Système

6.1 Séquence de Démarrage

Le processus de démarrage d'un système Linux suit une séquence précise :

6.1.1 Phase 1 : Matériel et Firmware

1. **Power-on** : Alimentation du système.
2. **POST** : Power-On Self-Test. Le BIOS/UEFI vérifie le matériel de base.
3. **Initialisation matérielle** : Détection et initialisation des composants matériels (CPU, RAM, disques, etc.)

6.1.2 Phase 2 : Bootloader

1. **Chargement du bootloader** : Le firmware charge le bootloader depuis le MBR (BIOS) ou la partition EFI (UEFI).
2. **GRUB** : GRUB (GRand Unified Bootloader) affiche un menu permettant de choisir le système d'exploitation ou le noyau à charger.
3. **Chargement du noyau** : GRUB charge le noyau Linux et l'`initramfs` en mémoire.
4. **Passage au noyau** : GRUB transfère le contrôle au noyau avec les paramètres de démarrage.

6.1.3 Phase 3 : Noyau

1. **Initialisation du noyau** : Le noyau initialise ses structures de données internes.
2. **Détection matérielle** : Le noyau détecte et initialise le matériel (processeurs, mémoire, périphériques).
3. **Chargement des modules** : Les modules du noyau nécessaires sont chargés depuis l'initramfs ou depuis le système de fichiers.
4. **Montage de la racine** : Le système de fichiers racine est monté. Si le disque racine est sur LVM, RAID, ou crypté, les modules nécessaires doivent être dans l'initramfs.
5. **Libération de l'initramfs** : Une fois la racine montée, l'initramfs peut être libéré de la mémoire.

6.1.4 Phase 4 : Espace Utilisateur

1. **Processus init** : Le noyau lance le processus init (PID 1), qui est le premier processus utilisateur.
2. **Systemd** : Sur les systèmes modernes, systemd remplace init et gère tous les services système.
3. **Initialisation des services** : Les services système sont démarrés selon leurs dépendances et leur configuration.
4. **Targets** : Systemd active les targets (équivalents des runlevels) pour mettre le système dans l'état désiré (multi-user, graphical, etc.)

6.2 Initramfs (Initial RAM Filesystem)

L'initramfs est un système de fichiers temporaire chargé en mémoire avant le montage de la racine :

- **Objectif** : Fournir les outils et modules nécessaires pour accéder au système de fichiers racine.
- **Contenu** : Scripts d'initialisation, modules du noyau (pilotes), outils système (mkfs, fsck, etc.)
- **Nécessité** : Essentiel si la racine est sur LVM, RAID, système de fichiers crypté, ou réseau (NFS).
- **Exécution** : L'initramfs exécute des scripts qui montent la racine, puis passe le contrôle au système réel.

6.3 Systemd : Gestionnaire de Services Moderne

Systemd est le remplaçant moderne de SysVinit :

6.3.1 Concepts Fondamentaux

- **Unités** : Systemd gère différents types d'unités : services, targets, mounts, sockets, timers, etc.
- **Dépendances** : Les unités peuvent avoir des dépendances (requires, wants, after) qui déterminent l'ordre de démarrage.

- **Parallélisation** : Systemd démarre les services en parallèle quand c'est possible, accélérant le boot.
- **Journalisation** : Journal intégré (journald) qui remplace syslog pour une meilleure intégration.

6.3.2 Targets vs Runlevels

Les targets systemd remplacent les runlevels traditionnels :

- **poweroff.target** : Équivalent du runlevel 0 (arrêt).
- **rescue.target** : Équivalent du runlevel 1 (mode maintenance).
- **multi-user.target** : Équivalent du runlevel 3 (multi-utilisateurs texte).
- **graphical.target** : Équivalent du runlevel 5 (multi-utilisateurs graphique).
- **reboot.target** : Équivalent du runlevel 6 (redémarrage).

7 Gestion des Paquets

7.1 Concept de Paquetage

Un paquetage est une archive contenant un logiciel et ses métadonnées :

- **Contenu** : Fichiers binaires, bibliothèques, fichiers de configuration, documentation.
- **Métadonnées** : Nom, version, description, dépendances, scripts d'installation.
- **Avantages** : Simplifie l'installation, la mise à jour, et la désinstallation. Gère automatiquement les dépendances.

7.2 Gestion des Dépendances

Les dépendances sont une caractéristique clé des systèmes de paquets :

- **Dépendances** : Un paquet peut nécessiter d'autres paquets pour fonctionner (bibliothèques, outils, etc.)
- **Résolution automatique** : Les gestionnaires de paquets (YUM, DNF, apt) résolvent automatiquement les dépendances.
- **Conflits** : Certains paquets peuvent être incompatibles entre eux. Le gestionnaire détecte et résout ces conflits.
- **Base de données** : Les métadonnées des paquets installés sont stockées dans une base de données (/var/lib/rpm/ pour RPM).

7.3 Formats de Paquets

7.3.1 RPM (RedHat Package Manager)

- **Format** : Format binaire utilisé par RedHat, Fedora, SUSE, CentOS.
- **Structure** : Archive cpio compressée avec en-tête contenant les métadonnées.
- **Outils** : rpm (bas niveau), yum/dnf (haut niveau avec résolution de dépendances).
- **Dépôts** : Les paquets sont stockés dans des dépôts (repositories) accessibles via HTTP, FTP, ou localement.

7.3.2 DEB (Debian Package)

- **Format** : Format utilisé par Debian, Ubuntu, Mint.
- **Structure** : Archive ar contenant les fichiers de contrôle et les données.
- **Outils** : dpkg (bas niveau), apt/aptitude (haut niveau).
- **Dépôts** : Repositories organisés par distribution et composants (main, contrib, non-free).

8 Réseaux et Communication

8.1 Modèle TCP/IP

Linux implémente complètement la pile de protocoles TCP/IP :

- **Couche Application** : Services réseau (HTTP, SSH, FTP, etc.)
- **Couche Transport** : TCP (connexion fiable) et UDP (sans connexion, rapide).
- **Couche Réseau** : IP (Internet Protocol) pour le routage des paquets.
- **Couche Lien** : Interfaces réseau (Ethernet, Wi-Fi, etc.)

8.2 Sockets

Les sockets sont l'interface de programmation pour la communication réseau :

- **Abstraction** : Les sockets permettent aux applications de communiquer via le réseau de la même manière qu'avec des fichiers.
- **Types** : Sockets TCP (stream), UDP (datagram), Unix domain sockets (communication locale).
- **Adressage** : Les sockets utilisent des adresses IP et des ports pour identifier les points de communication.

8.3 Interfaces Réseau

- **Configuration** : Les interfaces réseau sont configurées avec des adresses IP, masques de sous-réseau, et routes.
- **Activation** : Les interfaces peuvent être activées/désactivées dynamiquement.
- **Routage** : Le noyau maintient une table de routage pour déterminer comment acheminer les paquets.

Conclusion

Ce document a présenté les concepts théoriques fondamentaux qui sous-tendent le fonctionnement des systèmes Unix/Linux. Comprendre ces mécanismes internes est essentiel pour :

- Diagnostiquer efficacement les problèmes système.
- Optimiser les performances.
- Sécuriser correctement le système.

— Prendre des décisions éclairées lors de l'administration.

La maîtrise de ces concepts théoriques, combinée à la pratique des commandes et outils, forme la base d'une administration système compétente.